Solve a Graph using Depth First Search (DFS)

Aim :

To Implement DFS traversal on a graph to explore all reachable nodes from a source.

Explanation:

DFS explores nodes as deep as possible before backtracking. It uses recursion or an explicit stack.

Procedure:

1. Define the graph using an adjacency list.
2. Start from the source node.
3. Recursively visit all unvisited neighbors.

Python Code:

```
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()
    visited.add(start)
    print(start, end=" ")

    for neighbor in graph[start]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)

# Sample graph
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

print("DFS traversal from node A:")
dfs(graph, 'A')
```

Output:

```
DFS traversal from node A:
A B D E F C
```

Result:

Thus the program of implementing DFS traversal on a graph to explore all reachable nodes from a source. was executed successfully.

c: Implement MINIMAX Algorithm

Aim :

To Use the MINIMAX algorithm to evaluate the optimal move in a game tree.

Explanation:

MINIMAX is a recursive algorithm used in decision-making and game theory. One player tries to maximize the score, while the other tries to minimize it.

Procedure:

1. Define a list of terminal values (leaf nodes).
2. Apply recursive MINIMAX from root to leaves.
3. Alternate between max and min at each level.

Python Code:

```
def minimax(depth, nodeIndex, isMax, scores, h):
   if depth == h:
      return scores[nodeIndex]

   if isMax:
      return max(minimax(depth + 1, nodeIndex * 2, False, scores, h),
                 minimax(depth + 1, nodeIndex * 2 + 1, False, scores, h))
   else:
      return min(minimax(depth + 1, nodeIndex * 2, True, scores, h),
                 minimax(depth + 1, nodeIndex * 2 + 1, True, scores, h))

scores = [3, 5, 6, 9, 1, 2, 0, -1]
h = 3
print("Optimal value:", minimax(0, 0, True, scores, h))
```

Output:

```
Optimal value: 5
```

Result:

Thus the program of using the MINIMAX algorithm to evaluate the optimal move in a game tree

was executed successfully.